



《AI大模型Ragent项目》——用户点了停止生成集群里发生了什么

来自： 拿个offer-开源&项目实战



马丁

2026年05月14日 22:00

开篇引言

上一篇讲了流式生成的正常路径——LLM 一个字一个字吐内容，StreamCallback.onContent() 接住，StreamChatEventHandler 累积并通过 SseEmitterSender 推给前端，onComplete() 触发落库，最后推 finish + done 事件，前端关闭连接。整个过程一气呵成，从第一个 Token 到最后一个 Token，没有意外。

但现实中，用户经常中途点停止按钮。可能是问错了想换个问题，可能是答案已经看到关键部分不想等了，也可能是模型答偏了及时止损。ChatGPT、通义千问、Kimi 这些产品都有停止生成按钮，用户对这个交互已经很熟悉了。

问题是：停止生成不是前端单方面的事。

前端关掉连接只是断了接收端，后端的 LLM 流还在跑，Token 还在消耗，线程还在占用。更复杂的是，假设你部署了两台服务器——节点 A 和节点 B。用户的停止请求被 Nginx 分配到了节点 A，但 LLM 的流式推理跑在节点 B 上。节点 A 收到停止信号，但它本地根本没有这个任务，怎么把取消信号送到节点 B？节点 B 收到信号后，怎么打断正在 while 循环里一行一行读 LLM 响应的线程？已经生成了一半的内容要不要保存？SSE 连接怎么优雅关闭——先推 cancel 事件再推 done，还是直接断开？

这就是本篇要解决的问题。从前端按钮到集群广播，到 LLM 流中断，到资源释放——取消机制的完整链路。

前端点了停止，后端收到了什么

1. 前端的停止按钮

回忆一下上一篇讲的正常流程：前端通过 EventSource 连接 GET /rag/v3/chat，建立 SSE 长连接。连接建立后，后端会推一个 meta 事件，里面带着 conversationId 和 taskId。前端把这个 taskId 存下来——它就是后面停止时的凭证。

用户点停止按钮后，前端做两件事：

1. 调 POST /rag/v3/stop?taskId=xxx，告诉后端取消这个任务
2. 等待后端推回 cancel 事件，收到后再关闭 EventSource

注意，前端不是直接断开连接，而是先发停止请求，等后端推回确认事件后再断。这样做的好处是：后端有机会保存已生成的内容，推送一个带 messageId 的 cancel 事件，前端可以据此展示部分回复而不是一片空白。

2. 后端入口：Controller → Service → TaskManager

停止请求的入口很简单：

```
@RestController
@RequiredArgsConstructor
public class RAGChatController {

    private final RAGChatService ragChatService;

    /**
     * 停止指定任务
     */
    @IdempotentSubmit
    @PostMapping(value = "/rag/v3/stop")
    public Result<Void> stop(@RequestParam String taskId) {
        ragChatService.stopTask(taskId);
        return Results.success();
    }
}
```

@IdempotentSubmit 做幂等保护——用户可能紧张地连点好几下停止按钮，只处理一次就够了。

Service 层更简单，一行转发：

```
@Override
public void stopTask(String taskId) {
```

```
taskManager.cancel(taskId);
}
```

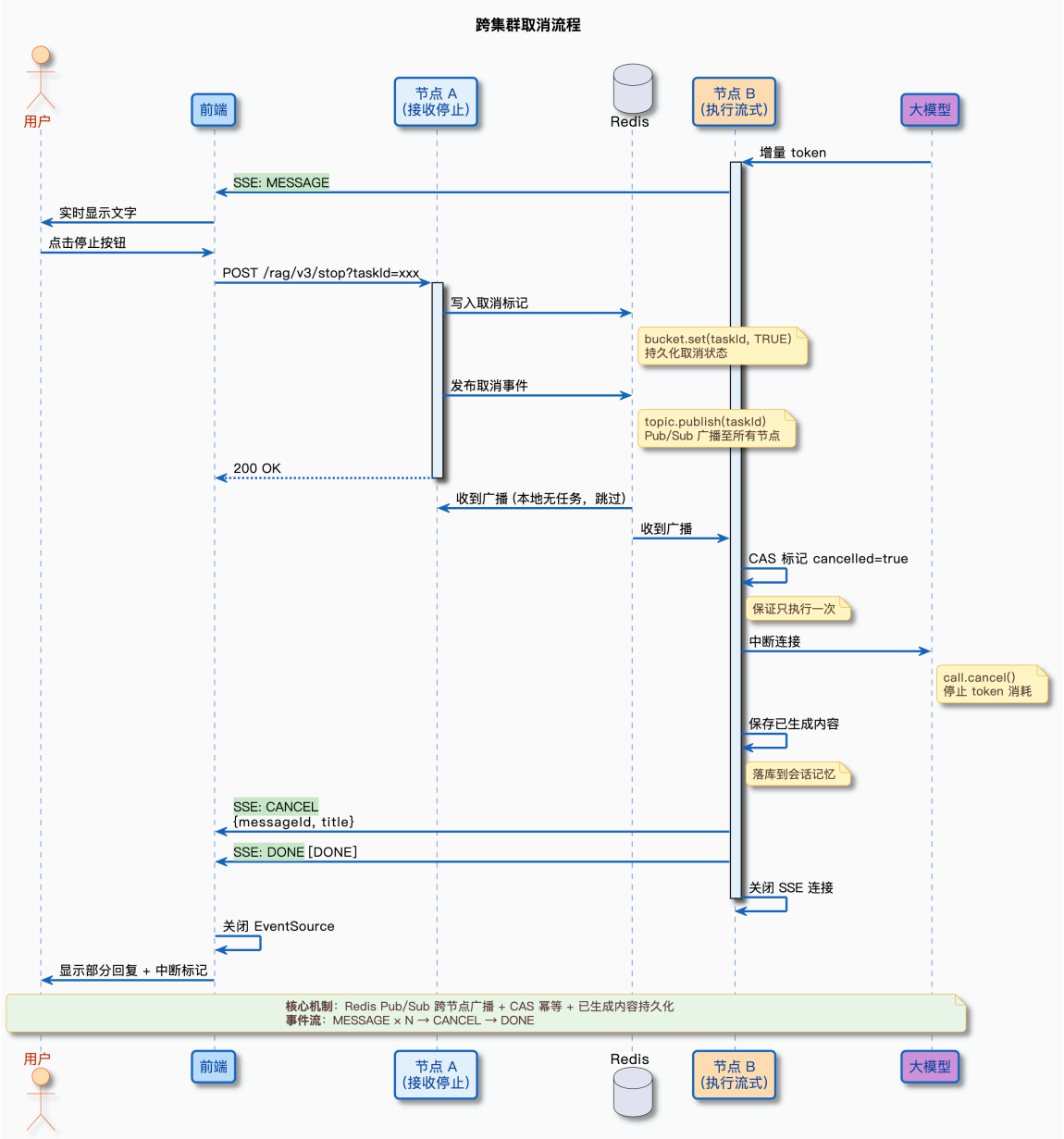
整条调用链：stop(taskId) → stopTask(taskId) → taskManager.cancel(taskId)。所有的复杂度都在 StreamTaskManager 里。

StreamTaskManager：跨集群任务管控的核心

StreamTaskManager 是本篇的主角。它要解决的核心问题是：在多机部署下，取消信号怎么从接收停止请求的节点传播到真正跑 LLM 流的节点，并且在各种时序竞争下都能正确工作。

1. 一张图看完整的取消流程

用一个具体场景贯穿：用户提了一个问题，LLM 正在节点 B 上流式生成回复。用户看到前几行发现答偏了，点击停止按钮，停止请求被负载均衡分配到了节点 A。



整个流程的关键在于：节点 A 不直接联系节点 B，而是通过 Redis 做中转。Redis 既做持久化标记（防止时序问题），又做 Pub/Sub 广播（实时通知）。节点 B 收到广播后在本地执行取消逻辑：中断 LLM 流、保存已生成内容、推 SSE 事件、关闭连接。

2. 两层存储：Guava Cache + Redis

StreamTaskManager 用了两层存储，各管各的事：

2.1 本地 Guava Cache

```
private final Cache<String, StreamTaskInfo> tasks = CacheBuilder.newBuilder()
    .expireAfterWrite(CANCEL_TTL) // 30 分钟自动过期
```

```
        .maximumSize(10000) // 最多存 1 万个任务
        .build();
```

Guava Cache 存的是 StreamTaskInfo——每个正在进行的流式任务在本地的注册信息：

```
private static final class StreamTaskInfo {
    private final AtomicBoolean cancelled = new AtomicBoolean(false);
    private volatile StreamCancellationHandle handle;
    private volatile SseEmitterSender sender;
    private volatile Supplier<CompletionPayload> onCancelSupplier;
}
```

四个字段分别是：取消标志位（AtomicBoolean，CAS 保证幂等）、LLM 流的取消句柄（用来中断 OkHttp 连接）、SSE 发送器（用来推事件和关连接）、取消时的回调（用来保存已生成内容并构造 CompletionPayload）。
为什么用 Guava Cache 而不是普通的 ConcurrentHashMap？两个原因：

expireAfterWrite(30 分钟)：任务完成后正常路径会调 unregister() 清理，但如果进程崩溃或者某些异常路径没走到清理逻辑，Guava Cache 的 TTL 会自动回收条目，不会内存泄漏
maximumSize(10000)：限制最大容量，即使出现异常情况导致大量任务堆积，也不会把内存撑爆

这两个能力是 ConcurrentHashMap 不具备的。

Guava Cache 的作用是管本地状态——这台机器上有哪些正在跑的任务，每个任务的取消句柄和 SSE 连接在哪里。读写都在本地内存，没有网络开销。这一点很重要，因为 StreamChatEventHandler 在每次 onContent() 回调中都要调 isCancelled() 检查任务是否已被取消。LLM 生成过程中这个检查可能每秒执行几十次，如果每次都查 Redis，延迟和负载都不可接受。

2.2 Redis 标记 + Pub/Sub

Redis 在取消流程中扮演两个角色：

RBucket（Key-Value 存储）——持久化取消标记

```
Key:    ragent:stream:cancel:{taskId}
Value:  true
TTL:    30 分钟
```

取消标记写入 Redis 后，30 分钟内任何节点都能查到这个任务已被取消。它解决的是时序问题——如果取消信号到达时任务还没注册到本地 Cache（比如 StreamChatEventHandler.initialize() 还没执行），等任务注册时可以去 Redis 查一下，发现已经被取消了，直接推 cancel 事件关连接。

RTopic（Pub/Sub）——实时广播取消信号

取消发生时，通过 RTopic.publish(taskId) 把 taskId 广播到所有订阅了 ragent:stream:cancel 频道的节点。每个节点收到后在本地查找并执行取消逻辑。注意取消方调 cancel() 时不区分本节点还是远端节点，广播会回到本机，统一由监听器处理，避免本地直调和远端调用走两条路径。

为什么两者都要用？因为它们各有短板：

Pub/Sub 是即时通知但不持久：Redis Pub/Sub 是 fire-and-forget，消息发出后不落盘、不重投。这里真正会丢的不是网络层面的消息丢失，而是消息到达节点时，对应的 taskId 还没走到 register() 那一步——订阅器收到 taskId，但本地 Cache 里查不到，这条广播对该任务就等于无效

Bucket 是持久化但不通知：写入 Redis 后 30 分钟内都能查到，但它不会主动通知各节点。如果只用 Bucket，就得靠轮询来发现取消状态，延迟太高

两者配合：Pub/Sub 负责实时通知已注册的任务立刻取消，Bucket 负责兜底未注册的任务在注册时发现自己已被取消。

3. cancel()：发起取消

```
public void cancel(String taskId) {
    RBucket<Boolean> bucket = redissonClient.getBucket(cancelKey(taskId));
    bucket.set(Boolean.TRUE, CANCEL_TTL);
    redissonClient.getTopic(CANCEL_TOPIC).publish(taskId);
}
```

两行代码，两步操作：

1. **先写标记**: `bucket.set(Boolean.TRUE, CANCEL_TTL)` 把取消标记写入 Redis, TTL 30 分钟
2. **再广播**: `topic.publish(taskId)` 通过 Pub/Sub 通知所有节点

顺序是先写标记再广播。这样做是为了防止一种极端情况: 如果先广播再写标记, 广播到达某个节点时, 该节点发现本地没有这个任务, 就忽略了; 紧接着任务注册到本地, 去 Redis 查标记——但标记还没写入 (虽然在同一个 Redis 实例中 Pub/Sub 和 SET 的时序很接近, 但语义上先写标记更稳妥)。

注意 `cancel()` 方法没有操作本地 Guava Cache。它只写 Redis 和广播。本地取消逻辑由 Pub/Sub 的监听回调 `cancelLocal()` 处理——即使取消请求打到的正好是跑任务的那台机器, 也是走广播再回到本地执行, 逻辑路径统一。

4. `subscribe()`: 监听取消频道

```
@PostConstruct
public void subscribe() {
    RTopic topic = redissonClient.getTopic(CANCEL_TOPIC);
    listenerId = topic.addListener(String.class, (channel, taskId) -> {
        if (StrUtil.isBlank(taskId)) {
            return;
        }
        cancelLocal(taskId);
    });
}

@PreDestroy
public void unsubscribe() {
    if (listenerId == -1) {
        return;
    }
    redissonClient.getTopic(CANCEL_TOPIC).removeListener(listenerId);
}
```

`@PostConstruct`——Spring 容器启动时自动订阅 `ragent:stream:cancel` 频道。每台服务器启动后都会订阅, 所以集群里每台机器都能收到取消广播。

收到消息后做两个判断: `taskId` 为空直接跳过 (防御性编程), 非空则调 `cancelLocal(taskId)` 在本地执行取消。

`@PreDestroy`——容器关闭时取消订阅, 优雅退出。`listenerId == -1` 说明 `subscribe()` 没成功 (比如 Redis 连接失败), 就不需要取消订阅。

5. `cancelLocal()`: 本地取消执行

这是取消流程中最关键的方法, 逐行拆解:

```
private void cancelLocal(String taskId) {
    // ① 从本地 Cache 查任务
    StreamTaskInfo taskInfo = tasks.getIfPresent(taskId);
    if (taskInfo == null) {
        return;
    }
    // ② CAS 保证只执行一次
    if (!taskInfo.cancelled.compareAndSet(false, true)) {
        return;
    }
    // ③ 中断 LLM 流
    if (taskInfo.handle != null) {
        taskInfo.handle.cancel();
    }
    // ④ 保存已生成内容 + 推事件 + 关连接
    if (taskInfo.sender != null) {
        CompletionPayload payload = taskInfo.onCancelSupplier.get();
        sendCancelAndDone(taskInfo.sender, payload);
        taskInfo.sender.complete();
    }
}
```

第①步：从 Guava Cache 查这个 taskId 对应的 StreamTaskInfo。查不到有两种可能：任务不在这台机器上（跑在别的节点），或者任务已经结束被清理了。无论哪种情况，直接 return，不做任何操作。这就是 Pub/Sub 广播的特点——所有节点都收到消息，但只有真正跑这个任务的节点会处理。

第②步：cancelled.compareAndSet(false, true) 做 CAS（Compare And Swap）操作。多个线程可能同时尝试取消同一个任务——比如 Pub/Sub 回调线程触发 cancelLocal() 的同时，register() 方法也检测到 Redis 中有取消标记并设置了 cancelled = true。CAS 保证只有一个线程成功把 false 改成 true，其他线程 CAS 失败直接返回。取消逻辑只执行一次。

第③步：调 handle.cancel() 中断 LLM 流。handle 可能为 null——如果取消发生在 bindHandle() 之前（LLM 还没开始调用），此时没有 handle 可以取消。不过没关系，bindHandle() 绑定时会检查 cancelled 标志，发现已取消会立刻调 handle.cancel()（下面第 6 节会详细讲）。

第④步：如果 sender 不为空（即 register() 已经执行过），做三件事：

```
onCancelSupplier.get(): 调取消回调，把已经累积在 answer 和 thinking 中的内容保存到数据库，返回包含 messageId 的
CompletionPayload

sendCancelAndDone(): 先推 cancel 事件（带 messageId 和 title），再推 done 事件

sender.complete(): 关闭 SSE 连接
```

执行顺序有讲究：先中断 LLM 流（第③步），再保存内容（第④步）。中断在前是为了尽快停止 Token 消耗；保存在后是因为内容已经不会再增加了（LLM 流已中断，onContent() 里的 isCancelled() 检查也会阻止新内容进入 answer），此时 answer.toString() 拿到的就是最终的部分回复。

6. 两个时序竞争点

取消信号可能在任务生命周期的任何阶段到达。StreamTaskManager 在两个关键位置做了时序检查，覆盖两种不同的竞争场景。

6.1 先取消后注册

场景：用户发起对话后极快地点了停止。此时流水线还在跑前面的阶段（比如意图识别、检索），StreamChatEventHandler.initialize() 还没执行，register() 还没调用。取消信号已经写入 Redis 并广播了，但本地 Cache 里还没有这个任务，cancelLocal() 查不到 StreamTaskInfo，直接 return 了。

如果不做任何处理，等任务注册时就会错过取消信号，LLM 流会正常跑到底。

register() 方法在注册时主动检查 Redis 标记：

```
public void register(String taskId, SseEmitterSender sender, Supplier<CompletionPayload> onCancelS
    StreamTaskInfo taskInfo = getOrCreate(taskId);
    taskInfo.sender = sender;
    taskInfo.onCancelSupplier = onCancelSupplier;
    if (isTaskCancelledInRedis(taskId, taskInfo)) {
        CompletionPayload payload = taskInfo.onCancelSupplier.get();
        sendCancelAndDone(sender, payload);
        sender.complete();
    }
```

```
    }  
}
```

注册完 `sender` 和 `onCancelSupplier` 后，立刻调 `isTaskCancelledInRedis()` 去 Redis 查取消标记。如果发现已被取消，当场执行取消逻辑——保存内容、推事件、关连接。不给 LLM 流启动的机会。

```
private boolean isTaskCancelledInRedis(String taskId, StreamTaskInfo taskInfo) {  
    if (taskInfo.cancelled.get()) {  
        return true;  
    }  
    RBucket<Boolean> bucket = redissonClient.getBucket(cancelKey(taskId));  
    Boolean cancelled = bucket.get();  
    if (Boolean.TRUE.equals(cancelled)) {  
        taskInfo.cancelled.set(true);  
        return true;  
    }  
    return false;  
}
```

这里先检查本地 `cancelled` 标志（快路径，不走网络），再查 Redis（慢路径，确认跨节点状态）。如果 Redis 标记为 `true`，同步设置本地标志。

这就是为什么 `cancel()` 方法要先写 Redis 标记再广播——标记先到位，即使广播被错过，`register()` 时查 Redis 也能发现。

6.2 先取消后绑定

场景：`register()` 已经执行完，但 LLM 还没开始调用（流水线在跑检索、Prompt 组装等步骤），没有 `StreamCancellationHandle`。此时取消信号到达，`cancelLocal()` 成功执行，`cancelled` 设为 `true`，但 `handle` 还是 `null`，第 ③ 步跳过了。随后流水线继续往前走，调用 LLM 拿到了 `handle`。

`bindHandle()` 方法在绑定时检查 `cancelled` 标志：

```
public void bindHandle(String taskId, StreamCancellationHandle handle) {  
    StreamTaskInfo taskInfo = getOrCreate(taskId);  
    taskInfo.handle = handle;  
    if (taskInfo.cancelled.get() && handle != null) {  
        handle.cancel();  
    }  
}
```

绑定 `handle` 后立刻检查——如果发现已取消，当场调 `handle.cancel()` 中断刚启动的 LLM 流。

这个方法在 `StreamChatPipeline` 中被调用：

```
private void streamRagResponse(StreamChatContext ctx, RetrievalContext retrievalCtx) {  
    IntentGroup mergedGroup = intentResolver.mergeIntentGroup(ctx.getSubIntents());  
    StreamCancellationHandle handle = streamLLMResponse(  
        ctx.getRewriteResult(),  
        retrievalCtx,  
        mergedGroup,  
        ctx.getHistory(),  
        ctx.isDeepThinking(),  
        ctx.getCallback()  
    );  
    taskManager.bindHandle(ctx.getTaskId(), handle);  
}
```

`streamLLMResponse()` 调 LLM 拿到 `handle`，紧接着 `bindHandle()` 绑定并检查。如果在这两行之间取消信号到达，`handle` 绑定时会立刻被取消。

两个检查点覆盖了不同阶段的时序竞争：

竞争场景	检查位置	检查方式
取消在 register() 之前	register() 内部	查 Redis 标记
取消在 register() 之后、bindHandle() 之前	bindHandle() 内部	查本地 cancelled 标志

7. isCancelled(): 回调中的轮询检查

```
public boolean isCancelled(String taskId) {
    StreamTaskInfo info = tasks.getIfPresent(taskId);
    return info != null && info.cancelled.get();
}
```

StreamChatEventHandler 在每次 onContent() 和 onThinking() 回调中都调这个方法：

```
@Override
public void onContent(String chunk) {
    if (taskManager.isCancelled(taskId)) {
        return;
    }
    answer.append(chunk);
    sendChunked(TYPE_RESPONSE, chunk);
}

@Override
public void onThinking(String chunk) {
    if (taskManager.isCancelled(taskId)) {
        return;
    }
    thinking.append(chunk);
    sendChunked(TYPE_THINK, chunk);
}
```

如果任务已被取消，直接 return——不再累积内容，也不再推送 SSE 事件。

这是取消机制的第三道防线。前两道是 register() 时查 Redis 和 bindHandle() 时查 cancelled，属于入口拦截；第三道是回调级别的轮询检查，属于过程拦截。即使 cancelLocal() 还没来得及调 handle.cancel() 中断 LLM 流，onContent() 里的检查也会阻止已取消任务的内容继续推送到前端。

注意 isCancelled() 只查本地 Guava Cache，不查 Redis。因为 LLM 生成过程中 onContent() 可能每秒触发几十次，每次都查 Redis 会严重拖慢流式推送。本地 Cache 的读取是纯内存操作，延迟在纳秒级。

8. unregister(): 正常完成时的清理

```
public void unregister(String taskId) {
    tasks.invalidate(taskId);
    redissonClient.getBucket(cancelKey(taskId)).deleteAsync();
}
```

正常完成时（onComplete() 中调用），做两件事：

- 1. tasks.invalidate(taskId)：从本地 Guava Cache 移除任务条目
- 2. deleteAsync()：异步删除 Redis 中的取消标记（如果有的话）

用 deleteAsync() 而不是 delete() 是有意为之——正常完成路径上不需要等 Redis 删除完再返回，异步删除减少延迟。即使异步删除失败了，Redis 标记也会在 30 分钟后 TTL 过期自动清理。

取消路径不走 unregister()。取消后，Guava Cache 中的条目靠 TTL 自动过期（30 分钟），Redis 标记也靠 TTL 过期。这是有意的设计——取消后可能还有延迟到达的回调需要查 isCancelled()，如果立刻清理了 Cache 条目，查到 null 会被当成正常状态，可能导致异常内容推送。

怎么中断正在跑的 LLM 流

取消信号到达 `cancelLocal()`，CAS 成功后，下一步就是调 `handle.cancel()` 中断 LLM 流。这个 `handle` 是怎么设计的？

1. StreamCancellationHandle 接口

```
public interface StreamCancellationHandle {  
    void cancel();  
}
```

一个方法的接口，职责单一：取消底层的流式操作。

为什么要抽象成接口而不是直接操作 `OkHttp` 的 `Call`？因为底层实现可能不只是 `OkHttp`。如果某天换用 `WebSocket` 或 `gRPC` 调用 LLM，只需要实现 `StreamCancellationHandle` 接口，上层的 `StreamTaskManager` 不需要改任何代码。

还有一个空实现 `NOOP`，用在异常场景（比如线程池拒绝执行时）：

```
public final class StreamCancellationHandles {  
  
    private static final StreamCancellationHandle NOOP = () -> {};  
  
    public static StreamCancellationHandle noop() {  
        return NOOP;  
    }  
  
    public static StreamCancellationHandle fromOkHttp(Call call, AtomicBoolean cancelled) {  
        return new OkHttpCancellationHandle(call, cancelled);  
    }  
}
```

2. OkHttpCancellationHandle：双通道取消

```
private static final class OkHttpCancellationHandle implements StreamCancellationHandle {  
  
    private final Call call;  
    private final AtomicBoolean cancelled;  
    private final AtomicBoolean once = new AtomicBoolean(false);  
  
    private OkHttpCancellationHandle(Call call, AtomicBoolean cancelled) {  
        this.call = call;  
        this.cancelled = cancelled;  
    }  
  
    @Override  
    public void cancel() {  
        if (!once.compareAndSet(false, true)) {  
            return;  
        }  
        if (cancelled != null) {  
            cancelled.set(true);  
        }  
        if (call != null) {  
            call.cancel();  
        }  
    }  
}
```

三个字段，各管一件事：

`once (AtomicBoolean)`：保证 `cancel()` 幂等。`cancelLocal()` 可能调一次，`bindHandle()` 发现已取消又调一次，用 CAS 确保实际只执行一次

`cancelled (AtomicBoolean)`: 协作式取消的标志位。设为 `true` 后, `doStream()` 的 `while (!cancelled.get())` 循环会在下一次迭代时退出。这是温和的取消方式——等当前行读完再退出

`call (OkHttp Call)`: 强制取消。`call.cancel()` 直接中断底层 TCP 连接。如果线程阻塞在 `source.readUtf8Line()` 上 (等待 LLM 返回下一个 Token), `call.cancel()` 会让 `readUtf8Line()` 立刻抛出 `IOException`, 线程从阻塞中醒来

两种取消方式互为补充:

如果线程正在循环的两行之间 (刚读完一行, 还没读下一行), `cancelled.set(true)` 足够——下一次 `while` 条件判断就退出了

如果线程阻塞在 `readUtf8Line()` 上等待 LLM 响应 (LLM 思考中还没吐下一个 Token), 光设标志位没用, 因为线程卡在 IO 上根本走不到 `while` 条件判断。此时 `call.cancel()` 从底层打断连接, 让阻塞调用立刻返回

双保险确保 LLM 流一定停得下来。

3. StreamAsyncExecutor: AtomicBoolean 的创建和传递

`OkHttpClientCancellationHandle` 用到的 `cancelled` 标志位是在 `StreamAsyncExecutor.submit()` 中创建的:

```
public final class StreamAsyncExecutor {

    private static final String STREAM_BUSY_MESSAGE = "流式线程池繁忙";

    static StreamCancellationHandle submit(Executor executor,
                                           Call call,
                                           StreamCallback callback,
                                           Consumer<AtomicBoolean> streamTask) {
        AtomicBoolean cancelled = new AtomicBoolean(false);
        try {
            CompletableFuture.runAsync(() -> streamTask.accept(cancelled), executor);
        } catch (RejectedExecutionException ex) {
            call.cancel();
            callback.onError(new ModelClientException(STREAM_BUSY_MESSAGE, ModelClientErrorType.SE
            return StreamCancellationHandles.noop();
        }
        return StreamCancellationHandles.fromOkHttp(call, cancelled);
    }
}
```

`cancelled` 的生命周期是这样的:

1. **创建**: 在 `submit()` 中 `new AtomicBoolean(false)` 创建
2. **传入读取线程**: 通过 `streamTask.accept(cancelled)` 传给异步任务, 异步任务内部把它传给 `doStream()` 作为 `while` 循环的退出条件
3. **传入取消句柄**: 通过 `StreamCancellationHandles.fromOkHttp(call, cancelled)` 传给 `OkHttpClientCancellationHandle`
4. **触发取消**: `handle.cancel()` 调用时, `cancelled.set(true)` 让读取线程的 `while` 循环退出

同一个 `AtomicBoolean` 对象在两个线程之间共享: 读取线程检查它, 取消线程设置它。这就是 Java 并发里经典的标志位协作取消模式。另外注意线程池拒绝的处理: 如果 `CompletableFuture.runAsync()` 抛 `RejectedExecutionException` (线程池满了), 直接取消 `OkHttp` 请求、回调 `onError()`、返回 `NOOP` 句柄。不会因为线程池满就让请求悬挂。

4. doStream(): 取消检测和异常区分

`doStream()` 是真真正正在线程池中执行的流式读取逻辑, 取消检测的核心就在这里:

```
private void doStream(Call call, StreamCallback callback, AtomicBoolean cancelled, boolean reasoni
    try (Response response = call.execute()) {
        if (!response.isSuccessful()) {
            String body = HttpResponseHelper.readBody(response.body());
            throw new ModelClientException(
                provider() + " 流式请求失败: HTTP " + response.code() + " - " + body,
                ModelClientErrorType.fromHttpStatus(response.code()),
                response.code()
            );
        }
    }
```

```

    }
    ResponseBody body = response.body();
    if (body == null) {
        throw new ModelClientException(provider() + " 流式响应为空", ModelClientErrorType.INVALID)
    }
    BufferedSource source = body.source();
    boolean completed = false;
    while (!cancelled.get()) { // ← 每读一行检查一次
        String line = source.readUtf8Line(); // ← 可能阻塞在这里
        if (line == null) {
            break;
        }
        if (line.isBlank()) {
            continue;
        }
        try {
            OpenAIStyleSseParser.ParsedEvent event = OpenAIStyleSseParser.parseLine(line, gson
            if (event.hasReasoning()) {
                callback.onThinking(event.reasoning());
            }
            if (event.hasContent()) {
                callback.onContent(event.content());
            }
            if (event.completed()) {
                callback.onComplete();
                completed = true;
                break;
            }
        } catch (Exception parseEx) {
            log.warn("{} 流式响应解析失败: line={}, provider(), line, parseEx);
        }
    }
    if (cancelled.get()) {
        log.info("{} 流式响应已被取消", provider());
        return; // ← 取消: 静默退出
    }
    if (!completed) {
        throw new ModelClientException(provider() + " 流式响应异常结束", ModelClientErrorType.INVALID)
    }
} catch (Exception e) {
    if (!cancelled.get()) {
        callback.onError(e); // ← 真正的异常才上报
    } else {
        log.info("{} 流式响应取消期间产生异常 (可忽略): {}", provider(), e.getMessage());
    }
    // ← 取消引起的异常: 只打日志
}
}
}

```

有三个关键点:

① while (!cancelled.get()): 协作式退出

每读一行之前检查 cancelled 标志。如果 OkHttpClientCancellationHandle.cancel() 设了 true, 下一次循环就会退出。退出后检查 cancelled.get(), 确认是取消导致的退出, 打 info 日志后直接 return——不调 onComplete(), 也不调 onError()。

② call.cancel() 导致的 IOException

当线程阻塞在 source.readUtf8Line() 上时, call.cancel() 会从底层中断 socket 连接, 导致 readUtf8Line() 抛出 IOException。这个异常会被外层 catch 捕获。

③ catch 块中区分取消和真正的异常

```

catch (Exception e) {
    if (!cancelled.get()) {
        callback.onError(e); // 真正的异常
    } else {

```

```
        log.info("...");    // 取消引起的异常，忽略
    }
}
```

如果 `cancelled` 为 `true`——说明是 `call.cancel()` 引起的 `IOException`，这不是错误，是预期行为。只打一行 `info` 日志，不调 `callback.onError()`。

如果 `cancelled` 为 `false`——说明是网络异常、LLM 服务端错误等真正的问题，调 `callback.onError()` 上报。

为什么取消场景不能调 `onError()`？因为 `cancelLocal()` 已经在走取消流程了——保存内容、推 `cancel` 事件、推 `done` 事件、关闭连接。

如果 `onError()` 也被触发，就会重复推送事件和关闭连接，可能引发异常。`StreamChatEventHandler.onError()` 里虽然也有 `isCancelled()` 检查做兜底，但在 `doStream()` 层面就把取消异常过滤掉是更干净的做法。

取消时的内容保存

1. buildCompletionPayloadOnCancel()

用户点停止时，LLM 可能已经生成了好几段内容。这些内容不应该丢弃——用户下次打开对话，应该能看到上次被中断的部分回复。

```
private CompletionPayload buildCompletionPayloadOnCancel() {
    String content = answer.toString();
    String messageId = null;
    if (StrUtil.isNotBlank(content)) {
        try {
            String thinkingContent = thinking.isEmpty() ? null : thinking.toString();
            ChatMessage message = ChatMessage.assistant(content, thinkingContent, resolveThinkingD
            messageId = memoryService.append(conversationId, userId, message);
        } catch (Exception e) {
            log.error("取消时持久化消息失败, conversationId: {}", conversationId, e);
        }
    }
    String title = resolveTitleForEvent();
    return new CompletionPayload(String.valueOf(messageId), title);
}
```

逻辑很直白：

1. 从 `answer` (`StringBuilder`) 取出已累积的内容
2. 如果有内容（不是空白），连同 `thinking` 内容一起构造 `ChatMessage`，调 `memoryService.append()` 持久化到会话记忆
3. 构造 `CompletionPayload` 返回，包含 `messageId` 和对话标题

两个细节值得注意：

落库失败只打 `error` 日志，不抛异常：`catch (Exception e)` 吞掉了异常。取消比保存重要——不能因为数据库异常导致取消流程卡住。最坏情况是用户下次看不到被中断的内容，但至少取消操作本身成功了，SSE 连接能正常关闭

thinking 内容一并保存：如果是深度思考模式，`thinking` 中可能也累积了推理过程。保存下来方便后续查看

这个方法被包装成 `Supplier<CompletionPayload>`，在 `register()` 时传入 `StreamTaskInfo`：

```
taskManager.register(taskId, sender, this::buildCompletionPayloadOnCancel);
```

`cancelLocal()` 中通过 `taskInfo.onCancelSupplier.get()` 调用，触发保存并获取载荷。

2. 前端怎么用 `cancel` 事件

```
private void sendCancelAndDone(SseEmitterSender sender, CompletionPayload payload) {
    CompletionPayload actualPayload = payload == null ? new CompletionPayload(null, null) : payload;
    sender.sendEvent(SSEventType.CANCEL.value(), actualPayload);
    sender.sendEvent(SSEventType.DONE.value(), "[DONE]");
}
```

先推 cancel 事件，再推 done 事件。
cancel 事件的载荷是 CompletionPayload：

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public record CompletionPayload(String messageId, String title) {}
```

前端收到 cancel 事件后可以做几件事：

- 如果 messageId 不为空，说明有部分内容被保存了，可以在 UI 上展示这条消息并标记为已中断状态
- 如果 messageId 为空，说明取消时还没有任何内容生成，不需要展示
- title 是对话标题，用于侧边栏的对话列表

done 事件是通用的结束信号，前端收到后关闭 EventSource。不管是正常完成还是取消，done 都是最后一个事件。

正常完成 vs 取消：对比

维度	正常完成	用户取消
触发方	LLM 输出完成标记 (finish_reason=stop)	用户点击停止按钮
内容完整性	完整回复	部分回复（可能只有几个字）
落库时机	onComplete() 中落库	buildCompletionPayloadOnCancel() 中落库
SSE 事件序列	finish → done	cancel → done
finish / cancel 载荷	{messageId, title}	{messageId, title} (messageId 可能为 null)
LLM 流	自然结束	被 handle.cancel() 中断
资源清理方式	unregister() 主动清理 Cache + Redis	Cache 条目靠 TTL 自动过期
Redis 标记清理	deleteAsync() 异步删除	TTL 30 分钟后自动过期
onComplete() / onError() 是否触发	onComplete() 正常触发	都不触发（被 isCancelled() 拦截）

前端可以通过监听 finish 和 cancel 两种不同的事件类型来区分结束原因。收到 finish 时展示完成状态，收到 cancel 时展示中断状态。但无论哪种，都等 done 事件到达后再关闭连接。

超时 / 报错 / 客户端断连的统一兜底

用户主动取消只是一种场景。现实中还有几种情况同样会导致 SSE 连接中断，需要后端正确处理。

1. SSE 超时

SseEmitter 在创建时设了超时时间：

```
SseEmitter emitter = new SseEmitter(ragDefaultProperties.getSseTimeoutMs());
```

如果在超时时间内没有数据推送（LLM 长时间不吐 Token，比如深度思考模式下模型在思考），SseEmitter 触发 onTimeout 回调 → SseEmitterSender 中的 closed.set(true) → 后续所有 sendEvent() 调用静默跳过（closed 为 true 直接 return）。

LLM 流本身还在跑，但推送通道已经关了。onContent() 里调 sendEvent() 发不出去但也不报错。最终 LLM 流结束走到 onComplete() 或 onError() 时，sender.complete() 因为 CAS 失败也会跳过。

2. 推送报错

sendEvent() 内部调 emitter.send() 时可能抛异常（比如客户端已断开但 Tomcat 还没触发回调）：

```
public void sendEvent(String eventName, Object data) {
    if (closed.get()) {
        return;
    }
    try {
        // ...
        emitter.send(SseEmitter.event().name(eventName).data(data));
    } catch (Exception e) {
        fail(e);
    }
}
```

catch 中调 fail() → closeWithError() → CAS 设 closed = true + emitter.completeWithError()。后续所有 sendEvent 调用直接跳过。

3. 客户端断连

用户关闭浏览器标签页、网络断开、或前端主动调 EventSource.close() 但没先发停止请求——Spring 检测到客户端断连后触发 onCompletion 回调 → closed.set(true)。效果和超时一样——推送通道关闭，LLM 流继续跑到结束，但推送的内容到不了前端。

4. 统一的保护逻辑

三种异常场景的共同点是：推送通道不可用了，但 LLM 流可能还在跑。SseEmitterSender 的 closed 标志位提供了统一的保护——不管是什么原因导致连接不可用，发送方只需要检查 closed 就行。不需要区分是超时、报错还是断连，处理逻辑完全一样：跳过发送，等 LLM 流自然结束。

注意，这些兜底场景下 LLM 流不会被主动中断——没有人调 handle.cancel()，Token 继续消耗。这是和用户主动取消的区别。如果想在客户端断连时也主动中断 LLM 流，需要在 SseEmitter 的 onCompletion / onTimeout 回调中调用 taskManager.cancel(taskId)。当前设计选择不这样做，可能是因为超时或短暂断连后前端可能重连，此时保留 LLM 流的结果更有价值。

SseEmitterSender 的线程安全设计

流式场景下，多个线程可能同时操作 SseEmitter：

LLM 的 onContent() 回调在线程池线程中执行，调 sendEvent() 推内容
onTimeout 回调在 Tomcat 线程中执行，设 closed = true
cancelLocal() 在 Redis Pub/Sub 的回调线程中执行，调 complete() 关连接

SseEmitter 本身不是线程安全的，必须在外面包一层保护。SseEmitterSender 就是这层保护：

```
@Slf4j
public class SseEmitterSender {

    private final SseEmitter emitter;
    private final AtomicBoolean closed = new AtomicBoolean(false);

    public SseEmitterSender(SseEmitter emitter) {
        this.emitter = emitter;
        emitter.onCompletion(() -> closed.set(true));
        emitter.onTimeout(() -> closed.set(true));
        emitter.onError(e -> closed.set(true));
    }

    public void sendEvent(String eventName, Object data) {
        if (closed.get()) {

```

```

        return;
    }
    try {
        if (eventName == null) {
            emitter.send(data);
            return;
        }
        emitter.send(SseEmitter.event().name(eventName).data(data));
    } catch (Exception e) {
        fail(e);
    }
}

public void complete() {
    if (closed.compareAndSet(false, true)) {
        emitter.complete();
    }
}

public void fail(Throwable throwable) {
    closeWithError(throwable);
    log.warn("SSE send failed", throwable);
}

private void closeWithError(Throwable throwable) {
    if (closed.compareAndSet(false, true)) {
        emitter.completeWithError(throwable);
    }
}
}

```

四层保护机制：

① 构造器注册三个回调

```

emitter.onCompletion(() -> closed.set(true));
emitter.onTimeout(() -> closed.set(true));
emitter.onError(e -> closed.set(true));

```

无论哪种原因导致连接结束，都第一时间设 `closed = true`。后续所有操作都能感知到连接已关闭。

② `sendEvent()` 发送前检查

```

if (closed.get()) {
    return;
}

```

发送前先检查。但要注意，检查和发送不是原子操作——可能检查时 `closed` 还是 `false`，发送时连接已经被另一个线程关了。所以还有 `catch` 兜底：发送抛异常时调 `fail()` 设 `closed` 并关闭连接。

③ `complete()` 用 CAS

```

if (closed.compareAndSet(false, true)) {
    emitter.complete();
}

```

多个线程可能同时尝试关闭连接——正常完成线程调 `complete()`，取消线程也调 `complete()`，超时回调线程触发 `onTimeout`。CAS 保证只有一个线程成功调 `emitter.complete()`，其他线程跳过。`emitter.complete()` 不是幂等的，重复调用会抛异常。

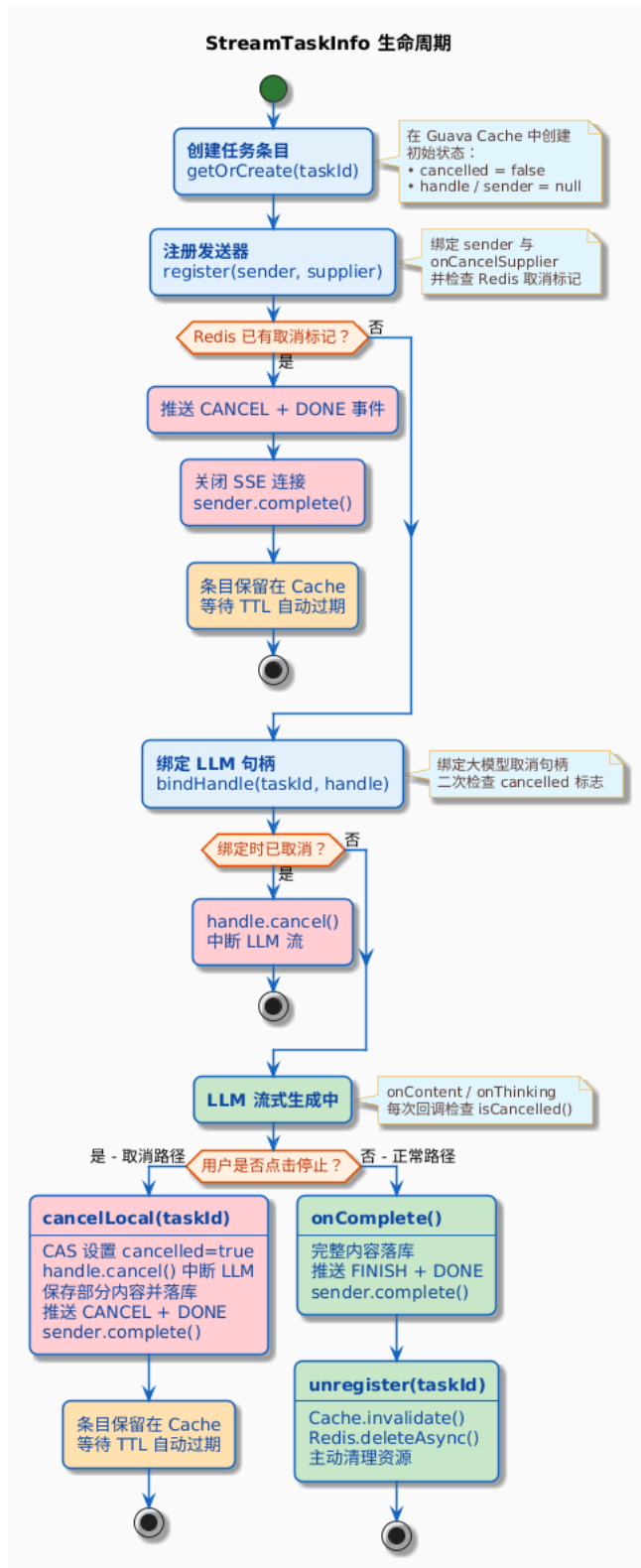
④ `closeWithError()` 也用 CAS

和 `complete()` 同理。`emitter.completeWithError()` 也不能重复调用，用 CAS 保护。

打个比方，SseEmitterSender 就像一扇旋转门——不管多少人同时推，门只转一次。AtomicBoolean closed 是这扇门的锁，CAS 是转门的动作。

StreamTaskInfo 的生命周期

一个 StreamTaskInfo 从创建到清理，经历以下阶段：



两条路径的清理方式不同：正常完成走 unregister() 主动清理两层存储；取消后不主动清理，靠 Guava Cache 的 TTL（30 分钟）和 Redis Key 的 TTL（30 分钟）自动回收。这是有意的设计——取消后保留 Cache 条目一段时间，让延迟到达的回调还能查到 isCancelled() 返回 true，避免取消后又有内容被推送出去。

小结与下一篇预告

回顾一下本篇的核心要点：

1. **取消入口**：前端调 `POST /rag/v3/stop?taskId=xxx`，后端一路转发到 `StreamTaskManager.cancel()`
2. **跨集群广播**：`cancel()` 先写 Redis 标记（持久化取消状态），再通过 Pub/Sub 广播到所有节点。接收节点在 `cancelLocal()` 中执行本地取消逻辑
3. **两层存储各司其职**：Guava Cache 管本地任务注册表，读写无网络开销，支撑高频的 `isCancelled()` 轮询检查；Redis 管跨节点状态同步，Pub/Sub 实时通知 + Bucket 持久化标记双管齐下
4. **CAS 保证幂等**：`cancelLocal()` 用 `compareAndSet` 确保取消逻辑只执行一次，`OkHttpClientCancellationHandle.cancel()` 也用 CAS 防重入
5. **双通道中断 LLM 流**：`cancelled.set(true)` 让 `while` 循环协作退出，`call.cancel()` 从底层中断 TCP 连接。双保险覆盖线程在循环中和阻塞在 IO 上两种状态
6. **两个时序检查点**：`register()` 时查 Redis 标记，覆盖先取消后注册的竞争；`bindHandle()` 时查本地 `cancelled` 标志，覆盖先取消后绑定的竞争
7. **取消时保存内容**：`buildCompletionPayloadOnCancel()` 把已累积的部分回复落库，前端收到 `cancel` 事件中的 `messageId` 后展示中断内容
8. `SseEmitterSender` **线程安全**：`AtomicBoolean closed` + 三个回调 + CAS 关闭，保护非线程安全的 `SseEmitter` 在多线程环境下正确工作

流式生成子系列到这里就收尾了。第 15 篇讲了正常路径——LLM 吐内容，回调累积推送，完成后落库关连接；第 16 篇补上了异常路径——取消信号跨集群广播，LLM 流被中断，部分内容被保存，SSE 连接优雅关闭。正常路径保证答案完整送达，异常路径保证资源不泄漏、用户不卡死。到这里，从第一个 Token 到最后一个事件，不管是跑完还是中断，整个流式生成的故事讲完了。

